

Test Automation Suite Agent (Multi-Agent AI System)

Architecture Plan (POC → Production-Ready Path)

> This document is a *system architecture plan* for a multi-agent AI system that generates an API test automation framework (TypeScript + Playwright), retrieves service knowledge using RAG (MongoDB Atlas Vector Search), and generates intelligent, context-aware API test cases. It is structured to be usable as an implementation blueprint.

1. Executive Summary

1.1 Objective

Build a **Test Automation Suite Agent** comprised of four cooperating agents/components:

1. **Orchestrator (Local CLI, POC)** – coordinates ingestion, scaffolding, and generation.
2. **Skeleton Framework Generator Agent** – produces a runnable Playwright API automation skeleton.
3. **Service Knowledge Retriever Agent (RAG)** – ingests enterprise sources into a vector store and retrieves grounded context.
4. **Intelligent Test Case Generator Agent** – generates Playwright API tests grounded in RAG + OpenAPI.

1.2 Key POC Decisions (validated)

- Language/framework: **TypeScript + Playwright** (`@playwright/test`, APIRequestContext)
- Orchestrator: **Local CLI** (interactive + non-interactive)
- LLM: **OpenAI** (cloud)
- Vector store: **MongoDB Atlas Vector Search**
- CI/CD target: **Jenkins** (integration via template/docs; POC runs locally)
- Repo integration: **Local generation only**, no Git auth in POC
- Jira: **Jira Cloud**
- Existing test assets: **Excel/CSV + Jira**
- Inputs supported: OpenAPI/Swagger, Confluence/Wiki docs (optional connector), Jira issues, Excel/CSV, Figma exports (+ optional Figma API), transcript text
- Code repo ingestion: **Skipped in POC**
- Incremental ingestion: **Required**

2. Architecture Overview

2.1 Logical Architecture (modules)

** (A) Orchestrator CLI **

- Owns run lifecycle and configuration.
- Invokes agents in deterministic phases.

** (B) Ingestion + RAG subsystem **

- Connectors fetch data from sources.
- Normalization + chunking pipelines.
- Embedding + indexing into MongoDB Atlas.
- Retrieval API used by Test Generator.

** (C) Generation subsystem **

- Scaffold generator produces framework baseline.
- Test generator produces endpoint suites based on OpenAPI + RAG.

2.2 Primary Data Flows

1. ****Ingest****: Sources → Normalize → Chunk → Embed → Store (Atlas)
2. ****Retrieve****: Query (endpoint/requirement) → Vector search + filters → ranked chunks + citations
3. ****Generate****: OpenAPI parse + retrieved context → structured test plan → code templates → Playwright tests

3. Inputs, Normalization, and Indexing Plan

3.1 Supported Inputs (POC)

Input	Example	Vectorization	Notes
-------	---------	---------------	-------

---	---	---	---
-----	-----	-----	-----

Requirements	BRD/User Stories (Jira, PDFs)	V	Jira Cloud connector first-class
--------------	-------------------------------	---	----------------------------------

UI specs	Figma exports PDF/images; optional Figma API	V	OCR best-effort
----------	--	---	-----------------

Recordings	Transcript text	V	chunk by paragraphs/speaker
------------	-----------------	---	-----------------------------

Existing tests	Excel/CSV and Jira	V	map to canonical schema
----------------	--------------------	---	-------------------------

API definition	OpenAPI/Swagger YAML/JSON	V	structured chunking by endpoint
----------------	---------------------------	---	---------------------------------

Tech docs	Confluence/Wiki/Swagger docs	V	optional connector
-----------	------------------------------	---	--------------------

Defects	Jira Cloud bugs	V	include repro + resolution
---------	-----------------	---	----------------------------

Release notes/test strategy	Docs/wiki	V	aids coverage targeting
-----------------------------	-----------	---	-------------------------

3.2 Canonical Document Schema

All source items normalize into:

- `sourceId` (stable)
- `sourceType` (openapi, jira_issue, excel_row, confluence_page, file_pdf, transcript)
- `uri/path`
- `timestampUpdated`
- `fingerprint` (sha256 of normalized content)
- `contentText` (plain text)
- `metadata` (projectKey, labels, endpoint, method, etc.)

3.3 Chunking Rules (deterministic)

- OpenAPI: chunk per `(path, method)` including parameters + responses + schemas.
- Jira: chunk per section (summary/description/AC/repro).
- Excel: chunk per row.
- Docs: chunk per heading.
- Transcript: chunk by paragraph/time window.
- Figma exports: OCR text by page/region + file structure context.

3.4 Incremental Ingestion Strategy (required)

- Maintain `sources` registry collection.
- Compare `fingerprint` to decide skip vs re-index.
- On change: delete old chunks for `sourceId`, re-chunk, re-embed, upsert.

4. MongoDB Atlas Vector Search Design

4.1 Collections

1. `sources`
 - Registry of ingested items with fingerprints.
2. `chunks`
 - One doc per chunk.
 - Fields:
 - `text`

- `embedding` (vector)
 - `sourceId`
 - `metadata` (docType, apiPath, method, jiraKey, tags)
3. `retrieval\_logs` (optional)

4.2 Vector Index

- Vector index on `chunks.embedding`
- Filterable metadata fields:
 - `metadata.docType`
 - `metadata.apiPath`
 - `metadata.httpMethod`
 - `metadata.jiraKey`

4.3 Retrieval Contract

- `retrieve(query, filters, k)` → chunks with citations
- `synthesize(query, filters)` → answer + citations

Grounding rules:

- Always return citations.
- If insufficient evidence, respond with “unknown” + missing-source guidance.

5. Agent Designs

5.1 Orchestrator Agent (Local CLI)

Responsibilities

- Gather configuration.
- Run phases:
 1. Ingest/refresh knowledge base
 2. Generate skeleton framework
 3. Generate tests
- Write run manifests and summaries.

Interfaces

- Commands:
- `tas-agent init`
- `tas-agent ingest`
- `tas-agent scaffold`
- `tas-agent generate`
- `tas-agent run`

Run Artifacts

- `.tas/manifests/run-<ts>.json`
- `.tas/cache/\*`

Extension Points

- Agent registry and connector registry (plugin pattern).

5.2 Skeleton Framework Generator Agent

Responsibilities

Generate a production-aligned baseline framework:

- Playwright config + TS config
- Environment config files
- API client factory + auth helper (API key)
- Common assertions and logging
- Playwright HTML report enabled

Output Structure

```
project/  
  package.json  
  playwright.config.ts  
  tsconfig.json  
  README.md  
  .env.example  
  config/dev.json  
  src/core/{config.ts,httpClient.ts,auth.ts,assertions.ts,logger.ts}  
  tests/api/health.spec.ts
```

Mocking Hook (optional)

- Config flag: `useMockServer: boolean`
- Placeholder integration points for Prism/WireMock/MSW.

5.3 Service Knowledge Retriever Agent (RAG)

Responsibilities

- Connectors: fetch and normalize sources.
- Chunking + embeddings.
- Persist chunks to MongoDB Atlas.
- Retrieve relevant knowledge for downstream agents.

Connectors (POC priority)

1. OpenAPI connector
2. Jira Cloud connector (requirements + defects)
3. Excel/CSV connector
4. Transcript connector
5. Figma export connector (OCR)
6. Confluence connector (optional)

5.4 Intelligent Test Case Generator Agent

Responsibilities

- Parse OpenAPI and enumerate endpoints.
- Retrieve endpoint context via RAG (requirements/defects/existing tests/release notes).
- Build structured ****Test Plan JSON**** per endpoint.
- Render Playwright spec files from templates.

Output Quality Rules

- Prefer OpenAPI-derived assertions (status codes, schema hints).
- Add negative tests only when constraints exist (required fields, auth).
- If unknown, generate TODO + citations rather than invent.

Traceability (basic)

- Header comment with Jira keys, OpenAPI operationId, and document references.

6. Integration Plan

6.1 Jira Cloud

- Auth: API token from ``.env``
- Data pulled: summary, description, acceptance criteria, repro steps.
- JQL configured in ``tas.config.yaml``.

6.2 Swagger/OpenAPI

- Parse YAML/JSON.
- Used for both indexing and generation.

6.3 Jenkins

- Provide sample Jenkins pipeline snippet.
- Archive Playwright HTML report.

7. Security, Governance, and Controls (POC)

- Secrets: ``.env`` locally; Jenkins credentials in CI.
- Avoid embedding secrets; optional redaction before embeddings.
- Local-only artifact generation.

8. Operational Plan (POC)

8.1 Developer Workflow

1. ``tas-agent init``
2. Configure ``tas.config.yaml`` and ``.env``
3. ``tas-agent ingest``

4. ``tas-agent scaffold --out ./repo``
5. ``tas-agent generate --out ./repo``
6. ``cd repo && npm i && TEST_ENV=dev npx playwright test``

8.2 Observability

- Console logs + run manifests.
- Optional retrieval logs.

9. Roadmap (POC → MVP)

- Add GitHub/Bitbucket PV ingestion (repo tree + PR metadata).
- Add Confluence ingestion if required.
- Add automated static checks (tsc/eslint) and optional gating.
- Add evaluation harness (golden endpoints, non-regression prompts).
- Add PR automation (branch + PR creation).

10. Recommended Tech Stack (to Build the Agent System)

This section describes the *implementation* tech stack for building the multi-agent system (Orchestrator + 3 sub-agents + RAG ingestion/retrieval).

10.1 Language & Runtime

- **Node.js 20+**
- **TypeScript 5+**

10.2 Agent / Orchestration Layer

Choose one of the following (POC can start simple, keep interfaces stable):

- **Option A (lean POC):** Custom agent interfaces in TypeScript (recommended for speed/control)
- ``Agent`` interface (``plan/execute/validate/artifacts``)
- ``Connector`` interface (``discover/fetch/normalize/fingerprint``)
- **Option B (framework-based):** LangGraph.js / LangChain.js
- Use for explicit agent graphs, retry policies, tool routing

10.3 LLM & Prompting

- **OpenAI API** (chat/completions)
- Prompt templating: **Handlebars** (or simple TS templates)
- Output conformance:
- **Zod** for schema validation of LLM outputs (Test Plan JSON, ingestion metadata)

10.4 Embeddings + Vector Store

- Embeddings: **OpenAI Embeddings** (configurable model)
- Vector DB: **MongoDB Atlas Vector Search**
- Mongo driver: **mongodb** official Node.js driver

10.5 Document Processing / Ingestion

- HTML → text: **cheerio** / **node-html-parser**
- PDF text extraction: **pdf-parse** (POC)
- Images/PDF OCR (best-effort):
- **tesseract.js** (local) OR
- call an external OCR service (if available)
- Excel/CSV parsing:
- **xlsx** (Excel)
- **papaparse** (CSV)

10.6 OpenAPI Tooling

- Parser/validator: **@apidevtools/swagger-parser**
- Types: **openapi-types**

10.7 Integrations / Connectors

- **Jira Cloud REST API** (via `fetch`/Axios)
- Optional later:
- Confluence REST API
- Figma API (metadata)

10.8 Test Framework Output Stack (Generated Repo)

- **@playwright/test**
- Node tooling:
- **npm** (or pnpm if preferred)

- **ts-node** only if needed (prefer compiled TS)
- Recommended dev quality (optional for POC, strong for MVP):
- **eslint** + **@typescript-eslint**
- **prettier**

10.9 Packaging & Distribution

- CLI framework: **commander** (or yargs)
- Build: **tsup** (fast TS bundling) or `tsc` + node`
- Distribution:
- npm package (private registry) or internal artifact

10.10 Observability (POC → MVP)

- POC: structured console logs
- MVP: **OpenTelemetry** traces + log export (optional)

Appendix: Minimal Config Sketch

```

serviceName: sample-service

llm:
  provider: openai
  model: gpt-4.1-mini
  embeddingsModel: text-embedding-3-large

vectorStore:
  provider: mongodb_atlas
  uriEnv: MONGODB_URI
  dbName: tas_agent

sources:
  openapi:
    - ./specs/openapi.yaml
  jira:
    baseUrlEnv: JIRA_BASE_URL
    emailEnv: JIRA_EMAIL
    apiTokenEnv: JIRA_API_TOKEN
    jql:
      - "project = ABC AND type in (Story, Bug) ORDER BY updated DESC"
  excel:
    - ./assets/existing-tests.xlsx
  transcripts:
    - ./assets/transcripts/grooming.txt
  figma:
    exports:
      - ./assets/figma/flows.pdf

generation:
  outputDir: ./generated-tests
  environments: [dev, qa, uat]
  auth:
    type: apiKey
    headerName: x-api-key
    valueEnv: API_KEY

```